



1 Option Pricing — Realized Variance Swap and Carr–Madan [9p]

Consider an underlying asset price process $(S_t)_{t \geq 0}$ under the risk-neutral measure $\mathbb{Q}$, given by

$$dS_t = rS_t dt + \sigma_t S_t dB_t, \quad (1)$$

where $(B_t)_{t \geq 0}$ is a standard Brownian motion and $(\sigma_t)_{t \geq 0}$ is an adapted stochastic volatility process. The money-market account is

$$A_t := e^{rt}, \quad t \in [0, T]. \quad (2)$$

We consider the realized variance over $[0, T]$, defined by

$$R_{0,T}^2 = \int_0^T \sigma_u^2 du. \quad (3)$$

1. Show, by applying Itô's formula to $\log S_t$, that

$$\int_0^T \sigma_u^2 du = 2 \int_0^T \frac{dS_u}{S_u} - 2 \log \left(\frac{S_T}{S_0} \right). \quad (4)$$

2. Define the time- t price process

$$\Pi_t := e^{-r(T-t)} \mathbb{E}_{\mathbb{Q}} \left[\int_0^T \sigma_u^2 du \mid \mathcal{F}_t \right], \quad t \in [0, T]. \quad (5)$$

Show that Π_t admits the decomposition

$$\Pi_t = L_t + 2(T-t)r e^{-r(T-t)} + 2e^{-r(T-t)} \int_0^t \frac{dS_u}{S_u}, \quad (6)$$

where

$$L_t := -2e^{-r(T-t)} \mathbb{E}_{\mathbb{Q}} \left[\log \left(\frac{S_T}{S_0} \right) \mid \mathcal{F}_t \right] \quad (7)$$

is the time- t price of the log-contract with payoff $-2 \log(S_T/S_0)$.

3. Show that the following portfolio, constructed at time $t \in [0, T]$, replicates the realized variance claim:

- one log-contract with value L_t ,
- ϕ_t shares of the underlying asset with positions

$$\phi_t := \frac{2e^{-r(T-t)}}{S_t} \quad (8)$$

- ψ_t units of the money-market account A_t

$$\psi_t := 2e^{-rT} \left(\int_0^t \frac{dS_u}{S_u} + (T-t)r - 1 \right) \quad (9)$$

4. Express L_t , and hence Π_t , in terms of prices of European call and put options on the underlying with maturity T . Explain how this is connected to the VIX estimator constructed in the first assignment. Discuss some advantages and disadvantages of this estimator, and of constructing the variance estimator from logarithmic returns rather than arithmetic returns.

Hint: Apply the spanning formula from Appendix A to the payoff $x \mapsto -2 \log(x/S_0)$.

5. Show that the portfolio in item 3 is self-financing.

6. Consider now the Heston variance process

$$dV_t = \kappa(\theta - V_t) dt + \sigma\sqrt{V_t} dW_t, \quad V_0 = v_0, \quad (10)$$

where $\kappa, \theta, \sigma > 0$. Compute the expected realized variance over the interval $[0, T]$, namely

$$\frac{1}{T} \mathbb{E}_{\mathbb{Q}} \left[\int_0^T V_t dt \right]. \quad (11)$$

Hint: Use Fubini's theorem:

$$\frac{1}{T} \mathbb{E}_{\mathbb{Q}} \left[\int_0^T V_t dt \right] = \frac{1}{T} \int_0^T \mathbb{E}_{\mathbb{Q}}[V_t] dt = \frac{1}{T} \int_0^T u(t) dt, \quad (12)$$

where $u(t) := \mathbb{E}_{\mathbb{Q}}[V_t]$. Derive an ODE for $u(t)$ and solve it.

7. Compute the fair variance swap rate

$$K_{\text{var}}(T) := \frac{1}{T} \mathbb{E}_{\mathbb{Q}} \left[\lim_{N \rightarrow \infty} \sum_{k=1}^N \left(\frac{S_{kT/N} - S_{(k-1)T/N}}{S_{(k-1)T/N}} \right)^2 \right] = \frac{1}{T} \mathbb{E}_{\mathbb{Q}} \left[\int_0^T \frac{1}{S_t^2} (dS_t)^2 \right], \quad (13)$$

for an index whose dynamics are given by the Heston-type model

$$\begin{cases} dS_t = (r - \alpha V_t) S_t dt + S_t \sqrt{\beta + V_t} dB_t, \\ dV_t = \kappa(\theta - V_t) dt + \sigma\sqrt{V_t} dW_t, \end{cases} \quad V_0 = v_0, \quad (14)$$

where B_t and W_t are standard Brownian motions with correlation $\rho \in [-1, 1]$.

Hint: Use the expression for $u(t) = \mathbb{E}_{\mathbb{Q}}[V_t]$ obtained in item 6.

2 The Temperature Derivative Market — Pricing of Asian-Style Temperature Risk [9p]

The majority of futures and options written on temperature indices are traded on the CME (Chicago Mercantile Exchange), while a significant portion of the market outside these standardized indices operates over-the-counter (OTC). Standardized exchange-traded contracts offer advantages such as reduced transaction costs, higher transparency, and centralized clearing, but they are limited to a relatively small number of temperature locations and contract designs.

The CME marketplace provides futures and options based on Heating Degree Days (HDD), Cooling Degree Days (CDD), and the Cumulative Average Temperature (CAT) index, covering a set of cities worldwide and contract seasons that depend on the specific product. Despite the availability of these contracts, trading volumes are often low and several days may pass without any transactions. Nevertheless, temperature derivatives remain an important laboratory for modeling non-tradable weather risk and for understanding how one can quantify, price, and hedge accumulated climatic exposures.

A weather derivative is a financial contract that facilitates the transfer of weather-related risk between parties seeking to hedge against such uncertainties. These contracts are typically structured around three core components:

- **Underlying meteorological parameter:** a physical atmospheric quantity monitored throughout the contract period to determine whether a payout is triggered. The most common parameter is average daily temperature, measured in degrees Celsius.
- **Weather index:** an aggregation of the underlying meteorological parameter over the contract period, such as HDD, CDD, or CAT.
- **Payoff function:** a predetermined rule that maps the realized weather index into a monetary payment.

While nearly any quantifiable weather risk can in principle be transferred by a weather derivative, some sectors are particularly exposed:

- **Energy companies** hedge demand-sensitive revenues against unusually mild or unusually severe temperatures.

- **Agricultural producers and transportation companies** manage losses generated by abnormal weather conditions.
- **Retail businesses** hedge seasonal demand fluctuations induced by temperature anomalies.
- **Leisure and tourism infrastructure** (hotels, amusement parks, ski resorts) depend strongly on favorable weather conditions.
- **Financial institutions** may use weather-linked claims as part of broader risk-transfer or portfolio-management strategies.

Temperature-Based Weather Derivatives

Temperature derivatives are by far the most widely used class of weather derivatives. This reflects the fact that temperature fluctuations materially affect many economic sectors. The underlying parameter is typically the average daily temperature, defined as the arithmetic mean of the daily maximum and minimum temperatures:

$$T_t := \frac{T_t^{\max} + T_t^{\min}}{2}. \quad (15)$$

From this daily temperature series one constructs the standard temperature indices:

$$HDD(\tau_1, \tau_2) := \int_{\tau_1}^{\tau_2} \max(c - T_u, 0) du, \quad (16)$$

$$CDD(\tau_1, \tau_2) := \int_{\tau_1}^{\tau_2} \max(T_u - c, 0) du, \quad (17)$$

$$CAT(\tau_1, \tau_2) := \int_{\tau_1}^{\tau_2} T_u du, \quad (18)$$

where c is a baseline reference temperature, typically 18°C , and T_u denotes average daily temperature on day u .

The integration period $[\tau_1, \tau_2]$ typically corresponds to:

- October to April for HDD contracts (heating season),
- April to October for CDD contracts (cooling season).

For a number of European cities, including Amsterdam, the CAT index is also actively used. Since

$$\max(T_u - c, 0) - \max(c - T_u, 0) = T_u - c,$$

one obtains the HDD–CDD parity

$$CDD(\tau_1, \tau_2) - HDD(\tau_1, \tau_2) = CAT(\tau_1, \tau_2) - c(\tau_2 - \tau_1). \quad (19)$$

Objective of the exercise. This exercise has four goals:

- retrieve and clean historical temperature data for Amsterdam,
- construct and calibrate a stochastic temperature model under the real-world measure $\mathbb{P}$,
- use the calibrated model to price options written on accumulated temperature indices,
- submit a fully standardized and reproducible solution so that all teams can be ranked on a common leaderboard.

2.1 Standardization and Competition Rules

This assignment has a competitive component. To make the comparison fair, **all teams must follow exactly the same data protocol, file naming convention, output format, and Monte Carlo settings.**

Fixed market and location. All teams must work with the same location:

Amsterdam, Netherlands, latitude 52.37, longitude 4.89.

Frequency and variable. For grading purposes, **only daily mean temperature** is used. The relevant series is `temperature.2m.mean`. Hourly data may be downloaded and explored, but it is not part of the graded output.

Time period and data split. All teams must use the same full sample and the same split:

Full sample: 2016-01-01 to 2026-02-01,
 Training set: 2016-01-01 to 2024-12-31,
 Test set: 2025-01-01 to 2026-02-01.

Implementation convention. Although the indices are written as integrals in continuous time, all pricing and backtesting in this assignment must be implemented on a **daily grid**. Therefore, for numerical work you must use

$$HDD(\tau_1, \tau_2) = \sum_{u=\tau_1}^{\tau_2} \max(18 - T_u, 0), \quad (20)$$

$$CDD(\tau_1, \tau_2) = \sum_{u=\tau_1}^{\tau_2} \max(T_u - 18, 0), \quad (21)$$

$$CAT(\tau_1, \tau_2) = \sum_{u=\tau_1}^{\tau_2} T_u. \quad (22)$$

Pricing convention. Weather markets are incomplete given that there is no available spot market for replication of contracts written on temperature futures. In this assignment, all teams must price under the **discounted expectation under their calibrated real-world model**:

$$\Pi_t = e^{-r(\tau_2-t)} \mathbb{E}^{\mathbb{P}}[\Pi \mid \mathcal{F}_t]. \quad (23)$$

The interest rate r for each contract will be provided in the contract file supplied.

Mandatory baseline model. For the graded baseline, every team must fit the following model:

- deterministic seasonal mean,
- AR(1) dynamics for deseasonalized residuals,
- deterministic seasonal volatility modeled by a Fourier series up to second harmonic.

2.2 Weather Data — Temperature

You may use the Open-Meteo API for downloading the temperature data. Documentation is available at [Open-Meteo GitHub API page](#).

Important: the graded submission must be based on the **daily mean temperature series for Amsterdam**. The historical forecast endpoint may be used for exploratory comparison, but the standardized cleaned dataset must use the exact full sample specified above.

Historical data:

```

1 import openmeteo_requests
2 import requests_cache
3 import pandas as pd
4 from retry_requests import retry
5
6 # Setup the Open-Meteo API client with cache and retry on error
7 cache_session = requests_cache.CachedSession(".cache", expire_after=3600)
8 retry_session = retry(cache_session, retries=5, backoff_factor=0.2)
9 openmeteo = openmeteo_requests.Client(session=retry_session)
10
11 # Historical archive endpoint
12 url = "https://archive-api.open-meteo.com/v1/era5"
13
14 # Amsterdam
15 params = {
16     "latitude": 52.37,
17     "longitude": 4.89,
18     "start_date": "2016-01-01",
19     "end_date": "2026-02-01",

```

```

20     "hourly": "temperature_2m",
21     "daily": "temperature_2m_mean"
22 }
23
24 responses = openmeteo.weather_api(url, params=params)
25 response = responses[0]
26
27 print(f"Elevation: {response.Elevation()} m")
28 print(f"Timezone: {response.Timezone()} {response.TimezoneAbbreviation()}")
29 print(f"UTC Offset (s): {response.UtcOffsetSeconds()}")
30
31 # Hourly data
32 hourly = response.Hourly()
33 hourly_temp = hourly.Variables(0).ValuesAsNumpy()
34
35 hourly_data = {
36     "date": pd.date_range(
37         start=pd.to_datetime(hourly.Time(), unit="s", utc=True),
38         end=pd.to_datetime(hourly.TimeEnd(), unit="s", utc=True),
39         freq=pd.Timedelta(seconds=hourly.Interval()),
40         inclusive="left"
41     ),
42     "temperature_2m": hourly_temp
43 }
44 hourly_dataframe = pd.DataFrame(data=hourly_data)
45
46 # Daily data
47 daily = response.Daily()
48 daily_temp_mean = daily.Variables(0).ValuesAsNumpy()
49
50 daily_data = {
51     "date": pd.date_range(
52         start=pd.to_datetime(daily.Time(), unit="s", utc=True),
53         end=pd.to_datetime(daily.TimeEnd(), unit="s", utc=True),
54         freq=pd.Timedelta(seconds=daily.Interval()),
55         inclusive="left"
56     ),
57     "temperature_2m_mean": daily_temp_mean
58 }
59 daily_dataframe = pd.DataFrame(data=daily_data)
60
61 print("\nHourly data:")
62 print(hourly_dataframe.head())
63
64 print("\nDaily data:")
65 print(daily_dataframe.head())

```

Forecast data (optional exploratory comparison):

```

1  import openmeteo_requests
2  import requests_cache
3  import pandas as pd
4  from retry_requests import retry
5
6  cache_session = requests_cache.CachedSession(".cache", expire_after=3600)
7  retry_session = retry(cache_session, retries=5, backoff_factor=0.2)
8  openmeteo = openmeteo_requests.Client(session=retry_session)
9
10 url = "https://historical-forecast-api.open-meteo.com/v1/forecast"
11 params = {
12     "latitude": 52.37,
13     "longitude": 4.89,
14     "start_date": "2016-01-01",
15     "end_date": "2026-02-01",
16     "hourly": "temperature_2m",
17     "daily": "temperature_2m_mean"
18 }
19
20 responses = openmeteo.weather_api(url, params=params)
21 response = responses[0]
22
23 print(f"Elevation: {response.Elevation()} m")
24 print(f"Timezone: {response.Timezone()} {response.TimezoneAbbreviation()}")
25 print(f"UTC Offset (s): {response.UtcOffsetSeconds()}")
26
27 daily = response.Daily()
28 daily_temp = daily.Variables(0).ValuesAsNumpy()
29

```

```

30 daily_data = {
31     "date": pd.date_range(
32         start=pd.to_datetime(daily.Time(), unit="s", utc=True),
33         end=pd.to_datetime(daily.TimeEnd(), unit="s", utc=True),
34         freq=pd.Timedelta(seconds=daily.Interval()),
35         inclusive="left"
36     ),
37     "temperature_2m_mean": daily_temp
38 }
39 daily_dataframe = pd.DataFrame(data=daily_data)
40
41 print(daily_dataframe.head())

```

Data cleaning rules. All teams must apply the following cleaning rules:

1. convert dates to ISO format YYYY-MM-DD,
2. keep exactly one row per calendar day,
3. if at most two consecutive daily observations are missing, fill them by linear or cubic interpolation,
4. if more than two consecutive observations are missing, keep the missingness flag and document it in the report,
5. store the final cleaned data in a file named `cleaned_daily_temperature.csv`.

2.3 Stochastic Modeling of Temperature Dynamics

We model temperature as the sum of a deterministic seasonal mean and a stochastic deviation around that mean. Let T_t denote the daily average temperature. We begin with the mean-reverting model

$$dT_t = \kappa(\mu(t) - T_t) dt + \sigma(t) dW_t, \quad (24)$$

where $W = (W_t)_{t \geq 0}$ is a standard Brownian motion, $\kappa > 0$ is the speed of mean reversion, $\mu(t)$ is the deterministic seasonal mean, and $\sigma(t) > 0$ is the volatility function.

To ensure that the model is centered exactly around the seasonal mean $\mu(t)$, we use the corrected dynamics

$$dT_t = (\mu'(t) + \kappa(\mu(t) - T_t)) dt + \sigma(t) dW_t. \quad (25)$$

If we define the deseasonalized process

$$X_t := T_t - \mu(t),$$

then (25) becomes

$$dX_t = -\kappa X_t dt + \sigma(t) dW_t. \quad (26)$$

Hence

$$X_t = e^{-\kappa t} X_0 + \int_0^t e^{-\kappa(t-u)} \sigma(u) dW_u. \quad (27)$$

Therefore,

$$T_t = \mu(t) + e^{-\kappa t} (T_0 - \mu(0)) + \int_0^t e^{-\kappa(t-u)} \sigma(u) dW_u. \quad (28)$$

In particular, if $T_0 = \mu(0)$, then

$$\mathbb{E}[T_t] = \mu(t). \quad (29)$$

Seasonal mean. We model the deterministic seasonal mean by

$$\mu(t) = a + bt + \alpha \sin(\omega t + \theta), \quad \omega = \frac{2\pi}{365.25}. \quad (30)$$

Equivalently, one may use the linearized representation

$$\mu(t) = a + bt + a_1 \cos(\omega t) + b_1 \sin(\omega t). \quad (31)$$

The two forms are related by

$$\alpha = \sqrt{a_1^2 + b_1^2}, \quad \theta = \text{atan2}(a_1, b_1). \quad (32)$$

Discrete-time baseline model. For the graded implementation, all teams must work on the daily grid. Let

$$X_i := T_i - \mu(i), \quad i = 1, 2, \dots$$

denote the deseasonalized daily temperature. The mandatory baseline residual model is

$$X_i = \phi X_{i-1} + \varepsilon_i, \quad |\phi| < 1. \quad (33)$$

The continuous-time mean-reversion speed is then recovered by

$$\kappa = -\log(\phi), \quad (34)$$

which is the exact OU mapping for daily step size $\Delta t = 1$.

Seasonal volatility. You must model the daily volatility as a deterministic Fourier series up to second harmonic:

$$\sigma(t) = v_0 + v_1 t + c_1 \sin(\omega t) + d_1 \cos(\omega t) + c_2 \sin(2\omega t) + d_2 \cos(2\omega t). \quad (35)$$

For implementation, if your fitted function becomes negative, you must truncate it as

$$\hat{\sigma}(t) := \max\{\sigma(t), 10^{-6}\}. \quad (36)$$

2.4 Estimate and Calibrate the Model under $\mathbb{P}$

Objective: calibrate the deterministic seasonal mean, fit the AR(1) residual dynamics, estimate the seasonal volatility, and save the results in a fully standardized way.

Task 1: Deterministic Seasonal Fit

1. Fit the seasonal mean in the form (31) on the training sample.
2. Estimate a, b, a_1, b_1 by least squares or `scipy.optimize.curve_fit`.
3. Save the fitted mean and residuals for the entire full sample.

Task 2: Residual Dynamics

1. Construct the deseasonalized series

$$X_i = T_i - \hat{\mu}(i).$$

2. Fit the AR(1) model (33) by OLS on the training set.
3. Compute

$$\hat{\kappa} = -\log(\hat{\phi}).$$

4. Report the estimated innovation standard deviation, AIC, and BIC.

Task 3: Seasonal Volatility

1. Compute the fitted residual innovations

$$\hat{\varepsilon}_i = X_i - \hat{\phi} X_{i-1}.$$

2. Construct an empirical daily volatility proxy by taking a centered 31-day rolling standard deviation of $\hat{\varepsilon}_i$.
3. Fit the Fourier volatility model (35) on the training set.
4. Compare the fitted volatility to the empirical rolling volatility on test samples.

Example code snippet

```
1 import numpy as np
2 import pandas as pd
3 from scipy.optimize import curve_fit
4
5 def model_fit(x, a, b, a1, b1):
6     omega = 2 * np.pi / 365.25
7     return a + b * x + a1 * np.cos(omega * x) + b1 * np.sin(omega * x)
8
9 daily_dataframe["date"] = pd.to_datetime(daily_dataframe["date"])
10 first_ord = daily_dataframe["date"].iloc[0].toordinal()
11
12 xdata = np.array([
13     d.toordinal() - first_ord
14     for d in daily_dataframe["date"]
15 ])
16
17 ydata = daily_dataframe["temperature_2m_mean"].to_numpy()
18
19 params_all, cov = curve_fit(model_fit, xdata, ydata, method="lm")
20
21 daily_dataframe["mu_hat"] = model_fit(xdata, *params_all)
22 daily_dataframe["residual"] = (
23     daily_dataframe["temperature_2m_mean"] - daily_dataframe["mu_hat"]
24 )
25
26 print("Estimated parameters:", params_all)
```

2.5 Pricing of Weather Derivatives

We now turn to the pricing of CAT and HDD derivatives. Since we work with daily temperature data, the relevant contract indices are discrete sums.

CAT index. For a contract period consisting of n days,

$$CAT_n = \sum_{i=1}^n T_i. \quad (37)$$

HDD index. Using the standard threshold 18°C ,

$$HDD_n = \sum_{i=1}^n \max\{18 - T_i, 0\}. \quad (38)$$

CAT call payoff.

$$\Pi_{\text{call}}^{CAT} = N \max\{CAT_n - K, 0\}, \quad (39)$$

where N is the tick size and K is the strike.

HDD call payoff.

$$\Pi_{\text{call}}^{HDD} = N \max\{HDD_n - K, 0\}. \quad (40)$$

General Asian-style temperature option payoffs. You may also consider the following standard forms:

- Call option with cap:

$$\xi = \min\{\alpha \max(DD - K, 0), C\},$$

- Put option with floor:

$$\xi = \min\{\alpha \max(K - DD, 0), F\},$$

- Collar:

$$\xi = \min\{\alpha \max(DD - K_1, 0), C\} - \min\{\beta \max(K_2 - DD, 0), F\},$$

where DD denotes the accumulated degree-day index over the contract period.

Pricing rule. For any contract with delivery period $[\tau_1, \tau_2]$ and payoff Π , the time- t model price is

$$\text{Price}_t = e^{-r(\tau_2-t)} \mathbb{E}^\mathbb{P}[\Pi \mid \mathcal{F}_t]. \quad (41)$$

Monte Carlo estimator. If M simulated paths are used, the estimator must be

$$\widehat{\text{Price}}_t = e^{-r(\tau_2-t)} \frac{1}{M} \sum_{m=1}^M \Pi^{(m)}, \quad (42)$$

with $M = 10,000$ for the standardized submission.

Simulation rule. Simulate future daily temperatures using the fitted baseline model:

1. simulate the AR(1) residual path X_i ,
2. combine it with the deterministic mean $\hat{\mu}(i)$,
3. use the fitted volatility $\hat{\sigma}(i)$ in the residual dynamics,
4. compute the temperature-index payoff on each path.

Variance reduction. If you use a variance-reduction method, you must document it in the report. However, the standardized pricing output must still use at least $M = 10,000$ reported discounted payoff samples.

2.6 Request for a Quote price

Let us assume that we have a request for a quote price which will be provided in the `contract_specifications.csv`. Your pricing code must read this file and compute prices for each contract it contains. The required columns in `contract_specifications.csv` are:

- `contract_id`
- `contract_type` (for example `CAT_CALL`, `HDD_CALL`, `CAT_FUT`)
- `city`
- `valuation_date`
- `start_date`
- `end_date`
- `strike`
- `notional`
- `baseline_c`
- `rate`

2.7 Evaluation Protocol and Competitive Ranking

The score consists of a reproducibility component and a competitive performance component.

Out-of-sample fit metrics. Let $\hat{T}_{u|u-1}$ denote the one-day-ahead forecast of temperature and let $\hat{\sigma}_u$ denote the fitted daily volatility. On the test set, we will compute

$$\text{RMSE}_T = \sqrt{\frac{1}{N} \sum_{u \in \mathcal{T}_{\text{test}}} (\hat{T}_{u|u-1} - T_u)^2}, \quad (43)$$

and

$$\text{RMSE}_\sigma = \sqrt{\frac{1}{N} \sum_{u \in \mathcal{T}_{\text{test}}} (\hat{\sigma}_u - \hat{\sigma}_u^{\text{emp}})^2}. \quad (44)$$

Pricing metric. Let P_j^{team} denote your submitted price for contract j , and let P_j^{bench} denote the benchmark price on the evaluation panel. The pricing metric is

$$\text{MAE}_{\text{price}} = \frac{1}{N_c} \sum_{j=1}^{N_c} |P_j^{\text{team}} - P_j^{\text{bench}}|. \quad (45)$$

A Assignment 1

Realized Variance Swaps - Another look at historical volatility We have already seen in the previous assignment some of the possible estimators one can construct for realized variance. Now our goal is to study realized variance swaps.

(Realized) variance swaps are forward contracts that allow for the exchange of the estimated volatility against a fixed value κ_σ . These constructs are typically written on the realized variance over a given period of time, i.e. have a terminal payoff of

$$\frac{1}{T} \left[\sum_{k=1}^N \left(\log \frac{S_{t_k}}{S_{t_{k-1}}} \right)^2 - \frac{1}{N-1} \left(\frac{S_T}{S_0} \right)^2 \right] - \kappa_\sigma^2 \quad (46)$$

and they can be priced using the expected value of their payoff

$$\frac{1}{T} \mathbb{E}_{\mathbb{Q}} \left[\sum_{k=1}^N \left(\log \frac{S_{t_k}}{S_{t_{k-1}}} \right)^2 - \frac{1}{N-1} \left(\frac{S_T}{S_0} \right)^2 \right] - \kappa_\sigma^2 \quad (47)$$

where κ_σ is the volatility level. Note that the above payoff has to be multiplied by the Vega notional¹, which is part of the contract, in order to convert it into currency units. Being a swap the strike is chosen such that the contract has no value at inception, i.e. $\kappa_\sigma^2 = \frac{1}{T} \mathbb{E}_{\mathbb{Q}} \left[\sum_{k=1}^N \left(\log \frac{S_{t_k}}{S_{t_{k-1}}} \right)^2 - \frac{1}{N-1} \left(\frac{S_T}{S_0} \right)^2 \right]$.

We can equivalently calculate the realized variance as follows:

$$\sigma_R^2 := A \sum_{k=1}^N \frac{1}{N} \log \left(\frac{S_{t_k}}{S_{t_{k-1}}} \right)^2 \quad (48)$$

where A is an annualization factor (usually 252 working days), $t_0 < t_1 < \dots < t_n$ are sampling dates specified in the contract and (S_t) denotes the stock price process under consideration. In practice however, pricing of variance swaps, namely computing the expectation of the realized variance, is performed by approximating the discrete sampling above, by its continuous version.

More precisely take a partition $0 = t_0 < t_1 < \dots < t_n = T$, then the following limit holds in probability:

$$\lim_{n \rightarrow +\infty} \sum_{k=1}^N \log \left(\frac{S_{t_k}}{S_{t_{k-1}}} \right)^2 = \langle \log(S), \log(S) \rangle_T \quad (49)$$

Computing the quadratic variation on the right is an easier exercise using the tools from stochastic Calculus and generally yields to simple closed-form expressions.

Spanning European payoffs

Consider a C^2 function $f : \mathbb{R} \rightarrow \mathbb{R}$ and some constant $F \geq 0$. By the fundamental theorem of calculus, we have

$$\begin{aligned} f(S) &= f(F) + \mathbf{1}_{\{S > F\}} \int_F^S f'(u) du - \mathbf{1}_{\{S < F\}} \int_S^F f'(u) du \\ &= f(F) + \mathbf{1}_{\{S > F\}} \int_F^S \left[f'(F) + \int_F^u f''(v) dv \right] du - \mathbf{1}_{\{S < F\}} \int_S^F \left[f'(F) - \int_u^F f''(v) dv \right] du \\ &= f(F) + f'(F)(S - F) + \mathbf{1}_{\{S > F\}} \int_F^S \int_v^S f''(v) du dv + \mathbf{1}_{\{S < F\}} \int_S^F \int_S^v f''(v) dv du \\ &= f(F) + f'(F)(S - F) + \mathbf{1}_{\{S > F\}} \int_F^S f''(v)(S - v) dv + \mathbf{1}_{\{S < F\}} \int_S^F f''(v)(v - S) dv \\ &= f(F) + f'(F)(S - F) + \mathbf{1}_{\{S > F\}} \int_F^\infty f''(v)(S - v)_+ dv + \mathbf{1}_{\{S < F\}} \int_0^F f''(v)(v - S)_+ dv \end{aligned} \quad (\text{A.0.1})$$

The following two cases are of particular financial importance:

¹Vega notional represents the expected profit and loss of a variance swap for a 1% change in volatility from the strike price. It helps traders assess the magnitude of the trade. For example, with a Vega notional of Eur10,000, a one-point difference in volatility would result in a profit or loss close to Eur10,000.

- if $F = 0$, then the expression above reduces to

$$f(S) = f(0) + Sf'(0) + \int_F^\infty f''(v)(S - v)_+ dv,$$

which means that the option with payoff $f(S)$ can be replicated by $f(0)$ invested in bonds, $f'(0)$ shares and an infinite strip of call options, each with strike v and in quantity $f''(v)$;

- if $F = S_0$, then the formula above reads

$$f(S) = [f(S_0) - S_0 f'(S_0)] + Sf'(S_0) + \mathbf{1}_{\{S > S_0\}} \int_{S_0}^\infty f''(v)(S - v)_+ dv + \mathbf{1}_{\{S < S_0\}} \int_0^{S_0} f''(v)(v - S)_+ dvdu,$$

so that the option with payoff $f(S)$ can be replicated with bonds, stocks and European calls and puts.